BRANDME

App Development Plan

A staged development roadmap for building Toronto's creator discovery platform

React Native | Expo SDK 54 | Supabase | Mapbox | PostHog

March 2026 · Updated Build Plan · Confidential

DOCUMENT OVERVIEW

Overview

BrandMe is a mobile-first social discovery platform for Toronto's creator community. It connects content creators, brand owners, artists, and anyone building an online presence with one another and with local events — think LinkedIn for creatives, built specifically for Toronto.

This document reflects the updated build plan following the Stage 6 refactor. The posts feed has been removed to keep the product focused on discovery and events. Portfolio images remain as a static showcase on profiles. The Create tab has been renamed **Host**. Events now carry their own niche associations via the **event_niches** table, making the niche filter system consistent across both Creator Mode and Event Mode on the map.

REFACTOR SUMMARY

What Changed

Removed	Posts feed on Home screen — Home is now a pure events feed
Removed	Post creation form on the Host tab — Host tab is events-only
Removed	posts table references from app code (table may remain dormant in DB)
Renamed	Create tab → Host tab (icon and label updated throughout)
Added	event_niches junction table — events declare their own niches directly
Added	Niche multi-select step in the Create Event (Host) form
Kept	portfolio_images table and portfolio grid on profiles
Kept	Supabase Storage — profile pictures, portfolio images, event cover images
Kept	follows / follower counts — core to creator discovery

GUIDING PRINCIPLES

Principles

Mock Data First, Real Data Later

All screens use typed mock data until the UI is stable. TypeScript interfaces mirror the DB schema so swapping in real Supabase calls is seamless. Authentication is the sole exception.

Direct Supabase Calls from the Frontend

Standard CRUD goes through @supabase/supabase-js. RLS policies handle authorization at the DB level. FastAPI is introduced only for complex geospatial queries or sensitive business logic.

One Stage, One Testable Milestone

Each stage ends with a clear checkpoint you can tap through on your phone. No stage depends on the next being complete.

Host Tab = Events Only

The Host tab has one job: creating an event. No content-type decision point. Simple and intentional.

Events Own Their Niches

Events declare their own niche associations via event_niches — independent of the host's profile niches. A networking event for musicians is discoverable as a music event regardless of who hosts it.

CONTENTS

Table of Contents

Stage 0	Project Scaffolding & TypeScript Foundation
Stage 1	Authentication & Onboarding
Stage 2	Account Page
Stage 3	Creator Map
Stage 4	Events System ✦ Updated
Stage 5	My Events & Event Management
Stage 6	Social Graph & Portfolio ✦ Updated
Stage 7	Go Live — Replace Mock Data with Supabase
Stage 8	Polish, Search & Discovery
Stage 9	Analytics, Testing & Launch Prep
Appendix	Database Schema Reference ✦ Updated

STAGE 0

Project Scaffolding & TypeScript Foundation

Creates the project skeleton and establishes conventions every future stage will follow. By the end you have a running Expo app with tab navigation and a complete type system that mirrors the database schema.

TASKS

#	Task	Details
1	**Initialize Expo project**	Run create-expo-app with the blank TypeScript template. Confirm the app runs on your physical device via Expo Go.
2	**Install core dependencies**	React Navigation (bottom tabs + native stack), TanStack Query, @supabase/supabase-js, react-native-maps. Do not configure Supabase yet.
3	**Build bottom tab navigator**	Five tabs: Home, Map, Host, My Events, Account. Each renders a placeholder screen. The Host tab uses a distinct icon (e.g. calendar-plus) to signal its purpose.
4	**Create /types directory**	TypeScript interfaces for: User, Profile ('CREATOR' | 'BUSINESS'), Niche, ExternalLink, Event, EventRSVP, Follow, PortfolioImage. No Post type. EventNiche link type included.
5	**Create /mock directory**	8–10 sample profiles, 5–6 events (each with niche associations), master niche list of 15–20 entries. Export as typed arrays.
6	**Set up project structure**	/src/screens, /src/components, /src/navigation, /src/types, /src/mock, /src/hooks, /src/utils, /src/constants. Add global theme/colors file.

✓ END-OF-STAGE CHECKPOINT

- App launches on your phone via Expo Go
- Five tabs visible — Home, Map, Host, My Events, Account
- No Post type exists in the type system; EventNiche type is defined
- Mock events include niche associations
- All TypeScript types compile without errors

STAGE 1

Authentication & Onboarding

The only stage where real Supabase infrastructure is set up before the UI is complete. Authentication is foundational — navigation guards, profile ownership, and RLS all depend on knowing who the current user is.

TASKS

#	Task	Details
1	**Create Supabase project**	Set up a new Supabase project. Enable email/password auth. Store URL and anon key in an environment config file (not hardcoded).
2	**Create database tables**	Create profiles, niches, profile_niches, external_links tables. Seed niches with 15–20 categories. Enable PostGIS extension. The event_niches table is created in Stage 4 alongside events.
3	**Configure Supabase client**	Initialize in /src/lib/supabase.ts using AsyncStorage for session persistence. Wrap app in TanStack QueryClientProvider.
4	**Build auth screens**	Welcome/Landing, Sign Up (email + password), Log In. Use Supabase auth.signUp() and auth.signInWithPassword(). Handle loading and error states.
5	**Build onboarding flow**	Multi-step after sign-up: (1) Creator or Business. (2) Display name and bio. (3) Niche multi-select chips. (4) Profile picture via Expo ImagePicker. (5) Location (device coords or manual). Write to profiles and profile_niches on completion.
6	**Add navigation guards**	Not logged in → auth stack. Onboarding incomplete → onboarding stack. Fully onboarded → main tab navigator.

✓ END-OF-STAGE CHECKPOINT

- New user can sign up and complete the full onboarding flow
- Profile data written to Supabase successfully
- Returning users land directly in the main app
- Logging out returns to Welcome screen
- Navigation guards prevent unauthenticated access

STAGE 2

Account Page

Built early to force a complete Supabase read/write loop and produce reusable components (ProfileCard, NicheTag) used across the map, events, and social features.

TASKS

#	Task	Details
1	**Build profile view screen**	Profile picture, display name, bio, profile type badge, years in business, niche tags, external links (tappable), follower/following counts (hardcoded 0 for now).
2	**Build profile edit screen**	Edit: display name, bio, years in business, profile picture, location, niches, external links. Save to Supabase on submit.
3	**Create reusable ProfileCard**	Compact card (picture, name, niche tags, profile type) for map bottom sheets, search results, follower lists.
4	**Create reusable NicheTag**	Styled chip/badge for niche labels. Used on profiles, event cards, and filter rows.
5	**Add Log Out button**	Calls Supabase auth.signOut() and returns to Welcome screen.
6	**Build View Other Profile screen**	Read-only profile view. Accepts profile ID as nav param, fetches from Supabase. Follow/Unfollow button (UI only for now).

✓ END-OF-STAGE CHECKPOINT

- Your own profile displays all onboarding data
- Every field can be edited and persists after reopening
- ProfileCard and NicheTag components render correctly
- Log out works and returns to Welcome screen
- View Other Profile renders when given a profile ID

STAGE 3

Creator Map

The app's signature feature. Users discover creators, businesses, and events near them. This stage focuses on Creator Mode using mock profile data. Event Mode is populated in Stage 4. The map is wired to Supabase in Stage 7.

TASKS

#	Task	Details
1	**Set up react-native-maps with Mapbox**	Install and configure react-native-maps. Obtain a Mapbox API key and configure a custom dark/branded map style. Center on Toronto by default.
2	**Render creator pins from mock data**	Plot mock profiles as custom markers. Distinct pin designs for Creators vs. Businesses. Implement marker clustering.
3	**Build bottom sheet for pin taps**	Tapping a pin slides up a bottom sheet with the ProfileCard. 'View Full Profile' button navigates to View Other Profile.
4	**Add mode toggle (Creator / Event)**	Toggle at the top of the map. Creator Mode shows creator/business pins. Event Mode is empty or shows 'Coming soon' until Stage 4.
5	**Add niche filter**	Horizontally scrollable niche chip row. Filters visible pins. State persists across mode switches. In Event Mode, filters by event_niches (active from Stage 4).
6	**Add search bar**	Filters pins by display name (Creator Mode) or event title (Event Mode). Persists across mode switches. Works in combination with niche filter.
7	**Handle location permissions**	Request on map load. If granted, show 'my location' dot. If denied, default to central Toronto view.

✓ END-OF-STAGE CHECKPOINT

- Map loads centered on Toronto with custom Mapbox style
- Creator and business pins visible with distinct styling
- Tapping a pin opens a bottom sheet with the ProfileCard
- Niche filter and search bar work and persist across modes
- Creator/Event mode toggle is present
- Current location dot appears if permissions granted

STAGE 4

Events System

Builds the complete events lifecycle: creating events, browsing the feed, RSVPing, and seeing events on the map. Events now carry their own niche associations — set during event creation and used to drive niche filtering independently of the host's profile.

TASKS

#	Task	Details
1	**Create events, event_rsvps & event_niches tables**	events: id, host_id, title, description, event_type enum (FASHION_SHOW | NETWORKING | WORKSHOP | LISTENING_PARTY | COMMUNITY | OTHER), privacy (PUBLIC | EXCLUSIVE), start_time, end_time, latitude, longitude, address_name, cover_image_url, status (DRAFT | PUBLISHED | CANCELLED), created_at. event_rsvps: event_id, profile_id, status (INTERESTED | REQUESTED | GOING | DECLINED), composite PK. event_niches: event_id (FK), niche_id (FK), composite PK — mirrors the profile_niches pattern.
2	**Build Create Event screen (Host tab)**	Form fields: title, description, event type picker, privacy toggle (Public / Exclusive), niche multi-select chips (same UI as onboarding — hosts tag what the event is for), date/time pickers, optional cover image upload (Supabase Storage), location picker. On submit: write to events table, insert rows into event_niches for each selected niche.
3	**Build Home screen (Events feed)**	Vertically scrollable feed of upcoming events using mock data. Each card shows: cover image, title, event type badge, privacy indicator, niche tags (1–2 visible), host name and picture, date/time, location, RSVP counts.
4	**Build Event Detail screen**	Full event view: cover image, all event info, niche tags, privacy badge, mini-map, host ProfileCard, Interested/Going attendee lists (mock), RSVP action buttons.
5	**Implement RSVP buttons**	Interested and Going on cards and detail screen. Toggle state stored locally for now (wired to Supabase in Stage 7). Exclusive events show 'Request to Attend' flow instead of direct Going.
6	**Add niche filter to feed**	Niche chip row filters the events feed by event_niches — not the host's profile niches. An event only appears when its own declared niches match the active filter.
7	**Populate Event Mode on the map**	Event Mode shows mock events as pins (calendar icon). Tapping opens a bottom sheet with event summary, niche tags, and 'View Details' button. Niche filter on the map also queries event_niches.

✓ END-OF-STAGE CHECKPOINT

- Create Event form includes niche multi-select and writes to event_niches on submit

- Event cards and Event Detail screen display the event's own niche tags
- Home screen shows a scrollable feed of mock events with niche tags visible
- Niche filter on the feed filters by event_niches, not host niches
- RSVP buttons toggle visually and store state locally
- Exclusive event flow shows Request to Attend
- Event Mode on the map shows mock event pins with niche-aware filter

STAGE 5

My Events & Event Management

Completes the events experience — viewing RSVPs, managing hosted events, and cancellations.

TASKS

#	Task	Details
1	**Build My Events screen**	Segmented control: Interested and Going. Each tab shows event cards filtered by RSVP status, pulled from local state.
2	**Add 'My Hosted Events' section**	On Account page: list of events the current user has created. Each card shows title, date, privacy badge, and RSVP counts. Tapping opens Event Detail.
3	**Build event edit functionality**	Host can edit title, description, time, location, privacy, cover image, and niches. Editing niches updates event_niches (delete existing rows, insert new ones). 'Cancel Event' sets status to CANCELLED.
4	**Add empty states**	Friendly empty state messages for Interested, Going, and Hosted tabs.

✓ END-OF-STAGE CHECKPOINT

- My Events screen shows separate Interested and Going lists
- Account page shows hosted events with RSVP counts and privacy badges
- Editing an event's niches correctly updates event_niches
- Cancelling an event marks it as cancelled in the feed
- Empty states display correctly in all tabs

STAGE 6

Social Graph & Portfolio

Adds the social layer (following, follower lists) and the portfolio showcase on profiles. Post creation and the posts feed are not part of this stage. Supabase Storage is used for profile pictures and portfolio images only.

Note: The posts table may remain dormant in Supabase — no app code references it.

TASKS

#	Task	Details
1	**Create follows table**	follower_id (FK), following_id (FK), created_at. Composite PK. Add follower_count and following_count as denormalized columns on profiles. DB trigger increments/decrements on insert/delete.
2	**Implement follow/unfollow**	Follow/Unfollow button on View Other Profile. Writes to follows table, updates denormalized counts. Button state reflects current relationship.
3	**Build follower/following lists**	Tapping counts on Account page opens a list of ProfileCards, each navigating to that user's profile.
4	**Set up Supabase Storage**	Confirm/create storage bucket. Access policies: users upload to their own folder; all authenticated users can read. Used for profile pictures and portfolio images.
5	**Build portfolio upload flow**	Profile owner uploads images via Expo ImagePicker. Images stored in Supabase Storage, portfolio_images record created with display_order. Support reordering and deletion.
6	**Add portfolio grid to profiles**	Grid of portfolio images below bio on Account page and View Other Profile. Tapping opens a full-screen lightbox.

✓ END-OF-STAGE CHECKPOINT

- Follow/unfollow works and counts update correctly
- Tapping counts shows a list of ProfileCards
- Portfolio images can be uploaded, reordered, and deleted
- Portfolio grid is visible on both own and other profiles
- No posts feed or post creation exists anywhere in the app

STAGE 7

Go Live — Replace Mock Data with Supabase

The UI is stable. This stage is a dedicated wiring pass replacing all mock data with real Supabase queries. TypeScript types already mirror the schema, so this is methodical rather than a rewrite.

TASKS

#	Task	Details
1	**Wire up the Creator Map**	Replace mock profile pins with Supabase PostGIS query (ST_DWithin via .rpc()). Fetch within visible map bounds only.
2	**Wire up the Events feed**	Upcoming events ordered by start_time, joined with host profile and event_niches data. Niche filter queries event_niches.
3	**Wire up RSVPs**	Move RSVP state to event_rsvps table. Buttons read/write Supabase. My Events queries event_rsvps by current user. Exclusive events write REQUESTED status.
4	**Wire up Event Mode on the map**	Event pins from Supabase PostGIS query filtered by map region and event_niches.
5	**Enable Row Level Security (RLS)**	RLS on every table. Users read all profiles and events, update only their own. Own RSVPs, follows, event_niches (for their events), and portfolio_images only. Test every policy.
6	**Add TanStack Query caching**	useQuery / useMutation for all Supabase calls. Profiles cache longer; events, RSVPs, follows refresh more frequently.
7	**Handle loading and error states**	Skeletons/spinners on every data-fetching screen. Error boundaries and retry logic. Pull-to-refresh on Events feed and My Events.

✓ END-OF-STAGE CHECKPOINT

- Every screen reads from and writes to Supabase — no mock data remains
- Map loads real profiles and events by geographic proximity
- Niche filter on feed and map queries event_niches correctly
- RSVPs and portfolio images persist in the database
- RLS policies tested and unauthorized writes rejected
- Loading states, error boundaries, and pull-to-refresh work throughout

STAGE 8

Polish, Search & Discovery

With all core features on real data, this stage focuses on quality and discoverability.

TASKS

#	Task	Details
1	**Build global search**	Search across creators (name, niche, bio) and events (title, description, location, event niches). Results in two sections: People and Events.
2	**Refine map interactions**	Smooth pin-selection animations. Bottom sheet swipe-to-dismiss and swipe-to-expand. 'Recenter on me' button. Cluster animations on zoom.
3	**Visual polish pass**	Consistent spacing, typography, and color throughout. Smooth screen transitions. Sharp on iOS and Android across screen sizes.
4	**Add push notifications**	Expo Notifications for: new follower, event starting soon (24h reminder), new event posted in a followed niche. Register push token with Supabase.
5	**Implement image caching**	expo-image or react-native-fast-image for profile pictures, portfolio images, and event cover images.
6	**Accessibility pass**	Accessibility labels on all interactive elements. VoiceOver and TalkBack testing. Sufficient color contrast.

✓ END-OF-STAGE CHECKPOINT

- Global search returns relevant creators and events including niche-based results
- Map interactions feel smooth with animations and expandable bottom sheets
- App is visually polished and consistent across all screens
- Push notifications delivered for follows and event reminders
- Images load fast and cache correctly
- App is usable with screen readers

STAGE 9

Analytics, Testing & Launch Prep

Prepares BrandMe for real users — analytics instrumentation, app store builds, and community testing in Toronto.

TASKS

#	Task	Details
1	**Integrate PostHog**	Track: screen views, profile views, event RSVPs, follows, portfolio views, niche filter usage, map interactions (zoom, pin taps, mode switches), search queries, onboarding completion rate.
2	**Set up EAS builds**	Configure eas.json for development, preview, and production profiles. Run preview build outside Expo Go to test full native experience.
3	**Create App Store assets**	App icon, splash screen, screenshots. App Store Connect and Google Play Console listings with description and metadata.
4	**Seed real data**	20–30 realistic test profiles across niches and Toronto locations. 10–15 upcoming events with cover images and niche tags. Data should feel like an active platform.
5	**Run community test**	Distribute to 10–20 Toronto creators via TestFlight and Android internal testing. Gather feedback on onboarding, map, event discovery, portfolio showcase. Monitor PostHog for drop-off.
6	**Bug fix and iterate**	Prioritize issues blocking core actions (sign up, find creators, RSVP, view portfolios) over cosmetic polish.

✓ END-OF-STAGE CHECKPOINT

- PostHog tracks all core user actions including niche filter usage
- App runs as a standalone build on iOS and Android
- App Store assets and metadata are ready
- Test data makes the app feel populated and active
- Community feedback gathered
- Critical bugs resolved
- App ready for first public release in Toronto

APPENDIX

Database Schema Reference

Complete table definitions listed in creation order (respecting foreign key dependencies). The posts table is excluded from the app and not listed here — it may remain dormant in Supabase.

profiles

- id — UUID, PK
- user_id — FK to auth.users.id
- display_name — text
- bio — text
- profile_type — enum: 'CREATOR' | 'BUSINESS'
- years_in_business — integer
- profile_picture_url — text (Supabase Storage)
- latitude, longitude — float (PostGIS coordinates)
- follower_count, following_count — integer, default 0 (denormalized)
- created_at — timestamp

niches

- id — PK
- name — text, unique (e.g. Streetwear, Cinematography, Barbering, Music Production)

profile_niches

- profile_id — FK to profiles.id
- niche_id — FK to niches.id
- Composite PK on (profile_id, niche_id)

external_links

- id — PK
- profile_id — FK to profiles.id
- platform_name — text (e.g. Instagram, TikTok, YouTube)

• url — text

events

• id — UUID, PK

• host_id — FK to profiles.id

• title — text

• description — text

• event_type — enum: FASHION_SHOW | NETWORKING | WORKSHOP | LISTENING_PARTY | COMMUNITY | OTHER

• privacy — enum: PUBLIC | EXCLUSIVE

• start_time, end_time — timestamp

• latitude, longitude — float

• address_name — text (e.g. 'The Liberty Village Commons')

• cover_image_url — text, nullable (Supabase Storage)

• status — enum: DRAFT | PUBLISHED | CANCELLED

• created_at — timestamp

event_niches

• event_id — FK to events.id

• niche_id — FK to niches.id

• Composite PK on (event_id, niche_id)

• Mirrors the profile_niches pattern — events declare their own niche associations directly, independent of the host's profile niches

event_rsvps

• event_id — FK to events.id

• profile_id — FK to profiles.id

• status — enum: INTERESTED | REQUESTED | GOING | DECLINED

• Composite PK on (event_id, profile_id)

follows

• follower_id — FK to profiles.id

• following_id — FK to profiles.id

• created_at — timestamp

• Composite PK on (follower_id, following_id)

• DB trigger increments/decrements denormalized counts on profiles

portfolio_images

• id — PK

• profile_id — FK to profiles.id

• image_url — text (Supabase Storage)

• display_order — integer

• created_at — timestamp

Note: The posts table is not listed. It may exist dormant in your Supabase project but is not referenced anywhere in the app codebase.